AI Call Assistant Updated Plan

Delivery plan

VERSION	STATUS	DATE
** 1.0	Product document	** 26 October 2026

This document describes the working prototype and the remaining configuration required for real inbound phone calls. External services are not represented as connected until verified.

1. Delivery objective

Turn the call-assistant plan into a runnable application with a local simulator first, then connect account-scoped cloud storage and a real inbound voice route when the owner supplies the required service accounts and phone-provider access. The local build is the first usable increment; it does not itself activate phone forwarding or third-party cloud services.

2. What is in this prototype now

- React/Vite dashboard with overview, call history, approved-instruction editor, and phone setup page.
- Node/TypeScript API with local call creation, caller turns, safe deterministic responses, transcript storage, and end-call summary.
- Optional Ollama chat integration with timeout and fallback.
- Firebase Google Sign-In client path, Firebase Admin ID-token verification, and UID-scoped Firestore call persistence.
- Twilio inbound voice and speech-turn webhooks with signature validation and an AI disclosure in the opening greeting.
- Firebase rules, `.env.example`, security notes, README, CI build workflow, PRD, SRS, and this plan.

3. Implementation stages

Stage	Work	Completion evidence
1. Usable local prototype	Start dashboard/API, create a test call, handle caller turns, save and review a summary.	Successful build and local API/UI smoke run.
2. Firebase account setup	Create Firebase project, enable Google sign-in, configure Firestore and private server credentials.	Authenticated sign-in and a saved record beneath the signed-in UID.
3. Voice provider setup	Provision a Twilio number or supported equivalent; deploy a public HTTPS API; set owner UID and signing token.	Valid signed webhook reaches the opening disclosure and completes speech turns.
4. Carrier routing	Set unanswered-call forwarding for the user's number to the provider number.	Controlled call to the user's number reaches the assistant only after no-answer routing.
5. Production readiness	Add user-managed instruction sync, retention/deletion/export controls, monitoring, alerting, and privacy/telecom review.	Reviewed production checklist and controlled release sign-off.

4. Configuration required from the system owner

- A Firebase project with Google Authentication and Firestore enabled.
- Firebase web configuration and a backend service credential stored outside Git.
- An Ollama endpoint/model if model-generated replies are desired; local deterministic replies work without it.
- A Twilio or compatible voice-provider account, phone number, and server-side webhook signing token.
- A public HTTPS deployment URL and permission to configure unanswered-call forwarding on the user's number.
- Confirmation of retention and disclosure expectations for the intended launch locations.

These are provider- and account-level prerequisites. The code cannot silently provision or claim them.

5. Local runbook

- Install Node.js 20+ and npm.
- Run `npm install`.
- Run `npm run dev` and open the local Vite address.
- Start a test call, confirm the first assistant turn discloses AI, enter a caller message, and end the call.
- Confirm the completed entry appears in call history. Local records are in ignored `work/data/demo-calls.json`.
- Run `npm run typecheck` and `npm run build` before pushing.

6. Current status and next technical work

Implemented in source

- Local call simulation and call history.
- Safety-oriented deterministic responses and optional Ollama.
- Firebase auth/API integration points and account-scoped Firestore writes.
- User-approved instructions stored in per-user Firestore settings when signed in.
- Signed Twilio-compatible inbound voice endpoints.
- Project documents and GitHub Actions build workflow.

Still required before a real phone call can be accepted

- Install packages and verify typecheck/build in the target environment.
- Configure actual Firebase web/admin credentials.
- Deploy publicly over HTTPS and verify Twilio signatures using the deployed URL.
- Set up provider number and call forwarding from the user's carrier.
- Place a controlled inbound call and verify the transcript and account record.

7. Release gates

- Local demo: install, build, health endpoint, create/turn/end flow, and call-history display all work.

- Cloud account: Google token verification and per-user isolation are verified with separate test accounts.
- Phone route: verified signature, AI disclosure, speech turn, safe fallback, hang-up completion, and record persistence are tested end to end.
- Production: environment secrets are private, HTTPS is enforced, local demo is disabled, retention/deletion is documented, and privacy/telecom requirements have been reviewed.

8. Project risks and controls

- **False claim of live answering:** distinguish demo/provider status in the dashboard and require an end-to-end call before marking ready.
- **Sensitive caller information:** keep records in user-scoped storage; add deletion and retention controls before production.
- **Model errors:** keep the assistant task narrow, use hard-coded emergency/sensitive-data safeguards, cap response time, and retain deterministic fallback.
- **Telephony changes or misrouting:** test with a number controlled by the owner and document how to disable forwarding immediately.
- **Drive unavailable:** Google Drive is not a dependency for call storage; any Drive backup remains a separate opt-in future capability.